

딥러닝이론및응용 기말 Project

학번: 20215254

이름: 최기현

Step 1: Dataset 준비하기

Data Preprocessing

```
[3]: # 경로 설정 (kaggle 사용시)
      # root_dir = '/kaggle/input/2025-dl-final/'
      # save_dir = '/kaggle/working/'

      # 경로 설정 (로컬 환경 사용시)
      root_dir = './2025-dl-final/'
      save_dir = './Model/'

[4]: train_dir = root_dir + 'train/'
      test_dir = root_dir + 'test/'

      train_csv_dir = root_dir + 'train.csv'
      test_csv_dir = root_dir + 'test.csv'

[5]: train_df = pd.read_csv(train_csv_dir)
      test_df = pd.read_csv(test_csv_dir)
      train_df.head(5)
```

사용한 데이터는 과제에서 제공된 총 12,500장의 예술 이미지와 각 이미지에 대한 메타 데이터입니다. 전체 데이터는 학습용 2,500장, 테스트용 10,000장으로 구성되어 있으며, 실제 이미지는 `train/` 및 `test/` 디렉토리에 저장되어 있고 이에 대한 메타데이터는 각각 `train.csv` 및 `test.csv`로 제공되었습니다. `train.csv`에는 각 이미지의 경로, 생성 원천 ID와 생성 원천을 나타내는 3가지 정답 레이블, 화풍과 화풍 ID를 나타내는 10가지 정답 레이블 정보가 포함되어 있고, `test.csv`에는 ID와 파일의 경로만이 포함되어 있습니다.

데이터를 준비하기 위한 코드는 기본적으로 제공되는 코드에서 로컬 환경에 사용할 수 있도록 일부 변형하였으며, `train_df` 와 `test_df`로 csv 파일을 로드하였습니다.

학습은 로컬 환경에서 수행되었으며, 학습환경 다음과 같습니다.

- Apple MacBook Air (Apple Silicon M3, 8-core CPU)
- Apple G16G 1,380 MHz (10-core GPU)

- 16GB RAM
- macOS Tahoe
- MPS (Metal Performance Shaders, Mac의 GPU 활용 백엔드)

Step 2: Dataset에 대한 Data Loaders 구성

데이터 전처리 구성(상) 및 데이터 로더 구성(하)

```
[10]: ## 학습 때 사용할 transform 정의 ##
mean, std = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

train_transform = A.Compose([
    A.HorizontalFlip(p=0.5),

    A.ShiftScaleRotate(
        rotate_limit=30, # 최대 30도까지 회전
        border_mode=cv2.BORDER_REFLECT_101,
        p=0.7
    ),

    A.RandomResizedCrop(
        size=(128, 128), # 이미지 사이즈
        scale=(0.8, 1.0), # 원본 대비 크기 범위
        ratio=(0.9, 1.1), # 가로 / 세로 비율 변경
        p=1.0
    ),

    A.Normalize(mean, std),
    ToTensorV2(),
])

## 평가용 transform 정의 ##
eval_transform = A.Compose([
    #
    #
    #
    A.Normalize(mean, std),
    ToTensorV2(),
])
```

```
[11]: num_workers = 0

# train / validation set 분리
train_df, valid_df = train_test_split(train_df, test_size=0.2, random_state=SEED)

train_dataset = CustomDataset(train_df, transform=train_transform, inference=False)
valid_dataset = CustomDataset(valid_df, transform=eval_transform, inference=False)

test_dataset = CustomDataset(test_df, transform=eval_transform, inference=True)

[info] Loaded 2000 rows
[info] Loaded 500 rows
[info] Loaded 10000 rows

[12]: batch_size = 128

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, pin_memory=True, num_workers=0)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False, pin_memory=True, num_workers=0)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, pin_memory=True, num_workers=0)
```

제공된 train.csv, test.csv를 기반으로, baseline.ipynb 에서 기본적으로 제공된, PyTorch의 Dataset 클래스를 상속한 CustomDataset을 사용하여 이미지와 레이블을 불러왔습니다. CustomDataset 클래스는 csv 파일 내부의 파일 경로를 통해 실제 이미지를 읽어온 후 BGR 채널을 RGB로 변환, 정의된 transform을 적용하여 학습에 사용할 (텐서, 원천 정답 레이블, 화풍 정답 레이블)로 반환하도록 구성되어 있습니다. 테스트 데이터의 경우 정답 레이블이 존재하지 않기에 이미지의 ID와 텐서만 반환합니다(inference=True 옵션).

데이터 전처리에 대한 설명

데이터 전처리는 제공된 baseline.ipynb에서 사용한 Albumentations 라이브러리를 그대로 사용하였습니다. 이미지 전처리는 함께 제공된 강의자료를 참고하여 다음과 같이 구성하였습니다.

- HorizontalFlip(p=0.5)
 - 50%의 확률로 이미지를 반전시킵니다.
- ShiftScaleRotate(rotate_limit=30, p=0.7)
 - 최대 30도 범위 내에서 회전시킵니다.
- RandomResizedCrop(size=(128, 128), scale=(0.8, 1.0), ratio=(0.9, 1.1), p=1.0)
 - 이미지의 일부 영역을 무작위 크롭한 뒤 원본 사이즈로 리사이즈합니다.
- Normalize(mean, std)
 - 이미지를 정규화 합니다. 평균과 표준편차값은 baseline.ipynb에서 제공된 값을 사용하였습니다.
- ToTensorV2()
 - 텐서 형태로 변환합니다.

학습 데이터와 검증 데이터의 분할은 `train_test_split`을 이용하여 2,500장 중 80%를 학습용, 20%를 검증용으로 사용하였습니다(`train_df`, `valid_df`). 각 데이터프레임을 `CustomDataset`에 전달하여 세 가지 데이터셋(`train_dataset`, `valid_dataset`, `test_dataset`)을 구성하고 이를 다시 PyTorch의 `DataLoader`에 감싸 배치 단위로 처리할 수 있도록 하였습니다. 배치 크기는 학습 과정중에 종종 변경되었지만, 최종적으로 128로 설정되었고, 학습용 데이터는 에폭마다 순서를 섞어주고 검증 및 테스트용 데이터는 고정된 순서를 유지시켰습니다.

Step 3: Neural Network 생성

<코드 캡처 첨부>

모델 생성자

```

class ArtClassifier(nn.Module):
    def __init__(self, drop_conv=0.25, drop_fc=0.5):
        super().__init__()

        # Convolutional layer
        self.conv1 = nn.Conv2d(3, 16, 3, stride = 1, padding = 1) # 3 * 128 * 128 -> 8 * 128 * 128
        self.conv2 = nn.Conv2d(16, 32, 3, stride = 1, padding = 1) # 8 * 64 * 64 -> 16 * 64 * 64
        self.conv3 = nn.Conv2d(32, 64, 3, stride = 1, padding = 1) # 16 * 32 * 32 -> 32 * 32 * 32
        self.conv4 = nn.Conv2d(64, 128, 3, stride = 1, padding = 1) # 32 * 16 * 16 -> 64 * 16 * 16
        self.conv5 = nn.Conv2d(128, 256, 3, stride = 1, padding = 1) # -> 256 * 8 * 8 = 8192

        # Max pooling layer
        self.pool = nn.MaxPool2d(2, stride=2)

        # Batch Normalization for conv
        self.conv1_bn = nn.BatchNorm2d(16)
        self.conv2_bn = nn.BatchNorm2d(32)
        self.conv3_bn = nn.BatchNorm2d(64)
        self.conv4_bn = nn.BatchNorm2d(128)
        self.conv5_bn = nn.BatchNorm2d(256)

        # Linear layer
        self.fc1 = nn.Linear(4096, 1024)
        self.fc2 = nn.Linear(1024, 256)
        self.fc3_source = nn.Linear(256, 3) # output for source
        self.fc3_style = nn.Linear(256, 10) # output for style

        # Dropout layer (p = 0.2 ~ 0.5)
        self.dropout_conv = nn.Dropout(drop_conv)
        self.dropout_fc = nn.Dropout(drop_fc)

        # ReLU
        self.relu = nn.ReLU(True)

        # Batch Normalization for FC
        self.fc1_bn = nn.BatchNorm1d(1024)
        self.fc2_bn = nn.BatchNorm1d(256)

```

```

def forward

```

```

def forward(self, x):

    # Convolutional and Max pooling layers
    x = self.pool(self.relu(self.conv1_bn(self.conv1(x))))
    x = self.pool(self.relu(self.conv2_bn(self.conv2(x))))
    x = self.pool(self.relu(self.conv3_bn(self.conv3(x))))
    x = self.pool(self.relu(self.conv4_bn(self.conv4(x))))
    x = self.pool(self.relu(self.conv5_bn(self.conv5(x))))

    # Resize image
    x = x.reshape(-1, 4096)

    # Add Dropout
    x = self.dropout_conv(x)

    # FC
    x = self.dropout_fc(self.relu(self.fc1_bn(self.fc1(x))))
    x = self.dropout_fc(self.relu(self.fc2_bn(self.fc2(x))))

    # Output layers
    x_source = self.fc3_source(x)
    x_style = self.fc3_style(x)

    return x_source, x_style

model = ArtClassifier(drop_conv=0.6, drop_fc=0.8)

model.to(device)

```

설계한 모델을 출력 후 네트워크를 구성한 방법과 이유를 각 단계별로 설명

과제의 목표인 두 가지 레이블 분류(그림 원천 및 화풍)를 동시예측하기 위해 위와 같은 Neural Network를 구성하였습니다. 입력되는 데이터는 RGB 채널 기준 (3 * 128 * 128)의 크기를 가지며, 5개의 Convolutional Layer와 2개의 FC 및 두 레이블에 대한 결과 레이어로 구성되어 있습니다.

Convolutional Layer는 순차적으로 3부터 256까지 늘어난 채널 수를 가지고, 모든 레이어는 커널 크기 3, 스트라이드 1, 패딩 1로 해상도를 유지하며 Max Pooling을 통해 단계마다 가로/세로 크기를 절반으로 줄이는 구조를 가집니다. 최종적으로 이미지는 평탄화되어 4096개의 feature로 변환됩니다. 이때, 각 Convolutional Layer 뒤에는 Batch Normalization과 ReLU가 적용됩니다. 이는 Layer별

출력 분포를 정규화하여 학습의 안정성을 높이고 에러를 최소화 하기 위함입니다.

ReLU 함수는 Convolutional Layer 이후 뿐만이 아닌 Fully Connected Layer에서도 사용되는데, ReLU 함수가 일반적으로 신경망 학습에 유리하다는 사실과 CNN 기반 이미지 분류에 가장 널리 사용되는 Activation Function으로 알려져 있어 이번 과제에서는 ReLU 함수를 사용했습니다.

평탄화 된 feature는 Fully Connected Layer에서 (4096 -> 1024 -> 256)와 같이 ¼크기로 줄어드는 구조를 사용하였습니다. FC에서도 마찬가지로 Batch Normalization과 ReLU를 사용하여 안정성을 높였습니다. 모델은 최종적으로 두 가지 출력 레이어를 가지게 됩니다. fc3_source는 (256->3)의 선형 레이어로, 이미지의 원천을 구분하기 위한 3가지 클래스 logit을 출력합니다. fc3_style은 (256->10)의 선형 레이어로, 이미지의 화풍을 구분하기 위한 10가지 클래스 logit을 출력합니다.

나아가, 모델에서의 정규화 기법 또한 포함하여, Convolutional Layer 이후 및 Fully Connected Layer의 레이어마다 각각 Dropout을 사용하였습니다. 이때 모델은 Convolutional Layer 이후의 드롭아웃 비율과 Fully Connected Layer 이후의 드롭아웃 비율을 모델 생성시마다 조절할 수 있도록 클래스 생성자의 매개변수로 전달할 수 있도록 구성하였습니다(drop_conv, drop_fc). 이를 통해 두 드롭아웃 비율의 다양한 조합을 실험하면서 over fitting을 억제하고 성능을 향상시키는 최적 드롭아웃 비율을 탐색할 수 있었습니다.

이번 과제에서는 시도하지 못했지만, 각 logit의 출력마다 Fully Connected Layer를 별도로 수행하도록 구성하여 더 높은 정확도를 얻을 수 있을것이라 생각합니다.

Step 4: Cost (Loss) Function과 Optimizer 선택

```
# from torch.optim.lr_scheduler import StepLR
from torch.optim.lr_scheduler import CosineAnnealingLR

criterion_source = nn.CrossEntropyLoss()
criterion_style = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr = 0.002, weight_decay = 0.0006)
# scheduler = StepLR(optimizer, step_size = 20, gamma = 0.5)
scheduler = CosineAnnealingLR(optimizer, T_max = 100, eta_min = 0.00001)
```

Optimizer와 Cost 함수를 선택한 이유와 선정하는데 중요하다고 생각하는 내용을 모두 작성합니다.

이 모델은 하나의 네트워크가 하나의 이미지에 대해 3가지 원천과 10가지 화풍으로 분류합니다. 모델의 두 가지 출력은 모두 클래스 분류를 목적으로 하기에 Cost Function으로 CrossEntropyLoss를 사용하였습니다. 각 두 가지 출력을 별개의 loss로서 구분하기 위해 원천 분류를 `criterion_source = nn.CrossEntropyLoss()`, 스타일 분류를 `criterion_style = nn.CrossEntropyLoss()`로 정의하고 학습 과정에서는 두 loss를 합하여 모델 전체의 loss로서 사용하였습니다. 이를 통해 두 분류 문제를 동시에 학습하도록 구성하였습니다.

Optimizer는 Adam을 사용하였습니다. Adam은 Momentum과 AdaGrad/RMSProp의 장점을 결합한 Optimizer로 각 학습의 방향, 파라미터 등에 따라 부드러운 학습 곡선을 보이며 SGD에 비교하여 그레디언트의 큰 변화에도 학습 속도를 적절하게 조절해 준다는 장점이 있습니다. Optimizer에 주어지는 하이퍼파라미터값은 제공된 자료의 예시 범위를 기반으로 lr을 설정하고 weight_decay L2 정규화를 추가하였습니다. 여러 하이퍼파라미터 조합을 실험한 결과, lr = 0.002, weight_decay = 0.00065 설정이 loss 감소와 정확도 상승 측면에서 가장 좋은 성능을 보여 결과적으로 선택하게 되었습니다.

제공된 강의자료에 소개된 Learning Rate Scheduler 기법을 참고하여 학습 진행에 따라 lr이 점진적으로 줄어들게 하는 기능을 포함시켰습니다. StepLR(optimizer, step_size=20, gamma=0.5)와 CosineAnnealingLR(optimizer, T_max=100, eta_min=0.00001) 두 가지 스케줄러를 시도했으며, 결과적으로는 CosineAnnealingLR을 사용하였습니다. 전체 학습 과정에서 lr이 서서히 감소하도록 하여 학습 후반에 작은 lr로 미세한 조정이 가능하도록 구성하였습니다. 스케줄링

사용은 실제 학습 과정에서 고정된 크기의 학습률을 사용했을 때와 비교하여 학습 초반에 큰 노이즈를 보였지만 후반에는 더 높은 일반화 성능을 보였습니다.

Step 5: 구성한 모델에 대한 Train and Validate 진행

단일 Train/Validation 학습	
<pre># Training code n_epochs = 150 train_loss = torch.zeros(n_epochs) train_acc_source = torch.zeros(n_epochs) train_acc_style = torch.zeros(n_epochs) valid_loss = torch.zeros(n_epochs) valid_acc_source = torch.zeros(n_epochs) valid_acc_style = torch.zeros(n_epochs) best_epoch = -1 best_val_loss = float('inf') best_val_src_acc = 0.0 best_val_sty_acc = 0.0 best_epoch_style = -1 best_acc_style = 0.0 best_acc_style_source_acc = 0.0 best_acc_style_loss = float('inf')</pre>	
모델 학습	모델 검증
<pre>for e in range(0, n_epochs): model.train() for data, label_source, label_style in train_loader: # 배치 샘플 가져오기 data, label_source, label_style = data.to(device), label_source.to(device), label_style.to(device) # gradient 초기화 optimizer.zero_grad() # 네트워크 결과 가져오기 logits_source, logits_style = model(data) # source, style 별 loss 계산 loss_source = criterion_source(logits_source, label_source) loss_style = criterion_style(logits_style, label_style) # 두 loss를 합해서 그레디언트 계산 loss = loss_source + loss_style loss.backward() # 옵티마이저 optimizer.step() # 예측별 loss 평균을 구하기 위해 loss저장 train_loss[e] += loss.item() # 예측별 accuracy 평균을 구하기 위해 저장(source) ps_source = F.softmax(logits_source, dim = 1) top_p_src, top_class_src = ps_source.topk(1, dim=1) equals = top_class_src == label_source.reshape(top_class_src.shape) train_acc_source[e] += torch.mean(equals.type(torch.float)).item() # 예측별 accuracy 평균을 구하기 위해 저장(style) ps_style = F.softmax(logits_style, dim = 1) top_p_stl, top_class_stl = ps_style.topk(1, dim=1) equals = top_class_stl == label_style.reshape(top_class_stl.shape) train_acc_style[e] += torch.mean(equals.type(torch.float)).item() train_loss[e] /= len(train_loader) train_acc_source[e] /= len(train_loader) train_acc_style[e] /= len(train_loader)</pre>	<pre>with torch.no_grad(): model.eval() for data, label_source, label_style in valid_loader: # 배치 샘플 가져오기 data, label_source, label_style = data.to(device), label_source.to(device), label_style.to(device) # 네트워크 결과 가져오기 logits_source, logits_style = model(data) # source, style 별 loss 계산 loss_source = criterion_source(logits_source, label_source) loss_style = criterion_style(logits_style, label_style) # 두 loss를 합 loss = loss_source + loss_style valid_loss[e] += loss.item() # 예측별 accuracy 평균을 구하기 위해 저장(source) ps_source = F.softmax(logits_source, dim = 1) top_p_src, top_class_src = ps_source.topk(1, dim=1) equals = top_class_src == label_source.reshape(top_class_src.shape) valid_acc_source[e] += torch.mean(equals.type(torch.float)).item() # 예측별 accuracy 평균을 구하기 위해 저장(style) ps_style = F.softmax(logits_style, dim = 1) top_p_stl, top_class_stl = ps_style.topk(1, dim=1) equals = top_class_stl == label_style.reshape(top_class_stl.shape) valid_acc_style[e] += torch.mean(equals.type(torch.float)).item() valid_loss[e] /= len(valid_loader) valid_acc_source[e] /= len(valid_loader) valid_acc_style[e] /= len(valid_loader)</pre>
결과 출력 및 저장	

```

if (valid_loss[e] < best_val_loss):
    best_val_loss = valid_loss[e].item()
    best_val_src_acc = valid_acc_source[e].item()
    best_val_sty_acc = valid_acc_style[e].item()
    best_epoch = e
    torch.save(model.state_dict(), 'best_model_statedict.pth')
    print(f">>> Best model updated at epoch {e}, val_loss={best_val_loss:.4f}")

if (valid_acc_style[e] > best_acc_style):
    best_acc_style_loss = valid_loss[e].item()
    best_acc_style = valid_acc_style[e].item()
    best_acc_style_source_acc = valid_acc_source[e].item()
    best_epoch_style = e
    torch.save(model.state_dict(), 'best_style_acc_statedict.pth')
    print(f">>> Best style acc model updated at epoch {e}, val_loss={best_val_loss:.4f}")

# lr 갱신
scheduler.step()
current_lr = scheduler.get_last_lr()[0]
print('lr: {:.6f}'.format(current_lr))

print('Epoch: {} \nTrain Loss: {:.6f} \tTrain Source accuracy: {:.6f} \tTrain Style accuracy: {:.6f}'.format(
    e, train_loss[e], train_acc_source[e], train_acc_style[e]))

print('Valid Loss: {:.6f} \tValid Source accuracy: {:.6f} \tValid Style accuracy: {:.6f} \n-----'.format(
    valid_loss[e], valid_acc_source[e], valid_acc_style[e]))

```

위 코드는 단일 Train/Validation 모델 학습 과정입니다. 분리한 train 데이터와 validation 데이터를 활용하여 특정한 하이퍼파라미터를 가진 모델의 학습 과정을 출력하고 결과를 그래프로 표현하여 확인할 수 있도록 구성하였습니다.

이전 단계에서 미리 정의한 Model, Optimizer를 활용합니다. 이 단계에서는 배치 단위로 데이터를 불러와 두가지 출력에 대한 loss를 계산하여($loss = loss_source + loss_style$) backward를 수행하고 Adam 옵티마이저를 통해 파라미터를 업데이트 합니다. 같은 epoch에서 validation 또한 마찬가지로 수행합니다. validation에서는 train에서 업데이트한 파라미터를 통해 loss와 두 가지 클래스에 대한 정확도를 측정하고 저장합니다. 이 과정은 에폭마다의 train/validation loss와 train/validation 의 이미지 원천 및 화풍 예측에 대한 정확도를 기록하여 학습 종료 이후 학습의 진행 과정을 그래프로 표현하여 확인할 수 있도록 작성했습니다. 이때 가장 낮은 validation loss를 보였을때와 가장 높은 validation의 화풍 정확도를 보였을때 두 가지를 기록하여 학습이 끝났을 때 확인할 수 있도록 하였습니다.

결과 그래프 예시



Step 6: CNN model training/validation 분석

KFold Validation 수행 코드 (일부)	
하이퍼파라미터 탐색 범위	파라미터별 KFold 반복
<pre> : # 최적 Hyperparameter 탐색 범위 지정 params = [# # 가장 낮은 valid loss를 보여줬었음 # {"name": "hp1_LR(0.003)_WD(0.0005)_Dropout_conv(0.5)_Dropout_fc(0.8)", # "lr": 0.003, "weight_decay": 0.0005, # "drop_conv": 0.5, "drop_fc": 0.8}, # # 여러 결과중 가장 많이 낮은 valid loss를 보임, 현재 가장 좋은 성능 # {"name": "hp1_LR(0.002)_WD(0.00065)_Dropout_conv(0.6)_Dropout_fc(0.8)", # "lr": 0.002, "weight_decay": 0.00065, # "drop_conv": 0.6, "drop_fc": 0.8}, # # 스타일 정확도 소폭 높음 # {"name": "hp2_LR(0.002)_WD(0.00065)_Dropout_conv(0.6)_Dropout_fc(0.75)", # "lr": 0.002, "weight_decay": 0.00065, # "drop_conv": 0.6, "drop_fc": 0.75}, {"name": "hp3_LR(0.002)_WD(0.0007)_Dropout_conv(0.6)_Dropout_fc(0.75)", "lr": 0.002, "weight_decay": 0.0007, "drop_conv": 0.6, "drop_fc": 0.75},] </pre>	<pre> num_epochs_per_fold = 125 # 파라미터별 결과저장 hp_results = [] # 가장 높은 sty_acc 추적 global_best_sty_acc = 0.0 global_best_sty_acc_info = None for hp_idx, hp in enumerate(params): print("\n\n=====") print("Parameter Set {}/{} -> {}".format(hp_idx+1, len(params), hp['name'])) # fold별 결과 저장 fold_results = [] for fold, (train_ind, valid_ind) in enumerate(kf.split(full_dataset)): print("\n\n==== Starting Fold {}/{} =====.format(fold+1, K)) # 데이터 분리 train_sampler_kfold = SubsetRandomSampler(train_ind) valid_sampler_kfold = SubsetRandomSampler(valid_ind) train_loader_kfold = torch.utils.data.DataLoader(full_dataset, batch_size=batch_size, sampler=train_sampler_kfold) valid_loader_kfold = torch.utils.data.DataLoader(full_dataset, batch_size=batch_size, sampler=valid_sampler_kfold) # 모델, 옵티마이저 초기화 model = ArtClassifier(drop_conv=hp["drop_conv"], drop_fc=hp["drop_fc"]).to(device) model.apply(reset_weights) </pre>

폴드별 학습 수행	KFold 결과 출력
<pre> for e in range(num_epochs_per_fold): model.train() for data, label_source, label_style in train_loader_kfold: # 배치 샘플 가져오기 data, label_source, label_style = data.to(device), label_source.to(device), label_style.to(device) # gradient 초기화 optimizer.zero_grad() # 네트워크 결과 가져오기 logits_source, logits_style = model(data) # source, style 별 loss 계산 loss_source = criterion_source(logits_source, label_source) loss_style = criterion_style(logits_style, label_style) # 두 loss를 합해서 그라디언트 계산 loss = loss_source + loss_style loss.backward() # 옵티마이저 optimizer.step() # 예측된 loss 평균을 구하기 위해 loss 저장 train_loss[i] += loss.item() # 예측된 accuracy 평균을 구하기 위해 저장(source) ps_source = F.softmax(logits_source, dim=1) top_p_src, top_class_src = ps_source.topk(1, dim=1) equals = top_class_src == label_source.reshape(top_class_src.shape) train_acc_source[i] += torch.mean(equals.type(torch.float)).item() # 예측된 accuracy 평균을 구하기 위해 저장(style) ps_style = F.softmax(logits_style, dim=1) top_p_stl, top_class_stl = ps_style.topk(1, dim=1) equals = top_class_stl == label_style.reshape(top_class_stl.shape) train_acc_style[i] += torch.mean(equals.type(torch.float)).item() </pre>	<pre> best_hp = min(hp_results, key=lambda d: d["mean_val_loss"]) print("\nBest parameters") print(" HP : {}".format(best_hp["hp"])) print(" Mean style acc : {:.6f}".format(best_hp["mean_val_sty_acc"])) print(" Mean source acc : {:.6f}".format(best_hp["mean_val_src_acc"])) print(" Mean val loss : {:.6f}".format(best_hp["mean_val_loss"])) print("\nBest style acc : {:.6f}".format(global_best_sty_acc)) print(" HP name : {}".format(global_best_sty_acc_info["hp_name"])) print(" HP idx : {}".format(global_best_sty_acc_info["hp_idx"])) print(" Fold : {}".format(global_best_sty_acc_info["fold"] + 1)) print(" Epoch : {}".format(global_best_sty_acc_info["epoch"])) print(" Val loss : {:.6f}".format(global_best_sty_acc_info["val_loss"])) print(" Src acc : {:.6f}".format(global_best_sty_acc_info["val_src_acc"])) print(" Sty acc : {:.6f}".format(global_best_sty_acc_info["val_sty_acc"])) </pre>

단일 Train/Validation 학습은 우연성이 존재할 가능성이 있기에 보다 안정적인 성능 평가와 최적 하이퍼파라미터 선택을 위해 KFold Validation을 사용하였습니다. 이를 위해 분리되어있던 train_dataset과 valid_dataset을 ConcatDataset을 통해 총 2500장의 full_dataset을 구성하였습니다. 전체 데이터는 5개의 fold로 나누었고(KFold(n_splits=5, shuffle=True)) 각 fold마다 SubsetRandomSampler를 활용하여 샘플을 분리하였습니다. KFold에 사용된 학습 기법은 단일 Train/Validation 학습에 사용된 기법과 동일합니다.

미리 정의해둔 params 리스트에 시도해 보고자 하는 하이퍼파라미터(Dropout 비율, weight decay, learning rate) 설정을 정의하고 각 설정에 대해 5번 KFold를 시도하여 각 fold별 최소 validation loss epoch(best_epoch), 해당 epoch의 loss 및 클래스별 정확도(best_val_loss, best_val_src_acc, best_val_sty_acc)를 기록하여 해당 fold의 평균 loss(mean_val_loss)를 기록, 평균 loss가 가장 낮은 모델의 설정을 채택하여 해당 fold의 하이퍼파라미터와 비슷한 값의 또 다른 하이퍼파라미터와 KFold를 시도하여 최적 파라미터를 찾아갔습니다.

또한, 화풍 정확도가 가장 높은 하이퍼파라미터도 별도로 추적하고 기록하여 가장 낮은 validation loss 및 가장 높은 style accuracy를 가진 설정을 비교할 수 있도록 구성하였습니다.

위에서 수행한 training + validation 과정을 설명하세요

1. 초기 설정

- wd 범위 - 0 ~ 0.00025

- Dropout 범위 - 0.25 / 0.5
- Learning Rate Scheduler 및 이미지 전처리 없음

Mean val loss : 1.3 ~ 1.5

매우 강한 overfitting 발생, Weight decay 및 Dropout 강도를 강화

2. Weight decay 및 Dropout 강화

- wd 범위 - 0.00025 ~ 0.0005
- Dropout 범위 - 0.5 / 0.7
- Learning rate 범위 - 0.001 ~ 0.005
- Learning Rate Scheduler 및 이미지 전처리 없음

Mean val loss : 1.05 ~ 1.15

wd 가 낮다면 Train loss는 거의 0에 가까워지지만 valid loss는 epoch이 40 이상일때 overfitting이 발생

3. Learning Rate Scheduler 및 이미지 전처리 추가, Batch size 조절

- 이미지 전처리 및 Learning Rate Scheduler 사용
- wd 범위 - 0.0005 ~ 0.0008
- Dropout 범위 - 0.6 / 0.7 ~ 0.8
- Learning rate 범위 - 0.0025 ~ 0.0035
- Batch size 범위 - 32 ~ 128

Mean val loss : 0.95 ~ 1.0

Learning Rate Scheduler 및 강한 정규화로 epoch이 늘어날수록 안정적으로 수렴

4. 최적 파라미터 선별

- wd - 0.00065
- Dropout - 0.65 / 0.75, 0.8
- Learning rate - 0.25
- Batch size - 128

Mean val loss : 0.9 ~ 0.95

다른 파라미터 대비 낮은 validation loss 및 5~10%정도 높은 정확도를 보임, dropout_fc가 0.75인 모델은 다른 모델 대비 뛰어난 화풍 정확도를 보이며, dropout_fc가 0.8인 모델은 validation loss가 뛰어남

Step 7: Predict with Test Data

최적 데이터 학습

<pre> # 학습 수행 for e in range(num_epochs): model.train() for data, label_source, label_style in full_loader: data, label_source, label_style = data.to(device), label_source.to(device), label_style.to(device) optimizer.zero_grad() logits_source, logits_style = model(data) loss_source = criterion_source(logits_source, label_source) loss_style = criterion_style(logits_style, label_style) loss = loss_source + loss_style loss.backward() optimizer.step() scheduler.step() torch.save(model.state_dict(), 'model_hp1_statedict.pth') print("Model saved") Model saved </pre>	
양상불용 모델 후보	제출 파일 생성
<pre> # 양상불용 모델 model1 = ArtClassifier(drop_conv=0.6, drop_fc=0.8).to(device) state1 = torch.load("model_hp1_statedict.pth", map_location=device) model1.load_state_dict(state1) model1.eval() model1_2 = ArtClassifier(drop_conv=0.6, drop_fc=0.8).to(device) state1 = torch.load("model_hp1_2_statedict.pth", map_location=device) model1.load_state_dict(state1) model1.eval() model1_3 = ArtClassifier(drop_conv=0.6, drop_fc=0.8).to(device) state1 = torch.load("model_hp1_3_statedict.pth", map_location=device) model1.load_state_dict(state1) model1.eval() model2 = ArtClassifier(drop_conv=0.6, drop_fc=0.75).to(device) state2 = torch.load("model_hp2_statedict.pth", map_location=device) model2.load_state_dict(state2) model2.eval() model2_2 = ArtClassifier(drop_conv=0.6, drop_fc=0.75).to(device) state2 = torch.load("model_hp2_2_statedict.pth", map_location=device) model2.load_state_dict(state2) model2.eval() model2_3 = ArtClassifier(drop_conv=0.6, drop_fc=0.75).to(device) state2 = torch.load("model_hp2_3_statedict.pth", map_location=device) model2.load_state_dict(state2) model2.eval() models = [model1, model2] </pre>	<pre> with torch.no_grad(): pbar = tqdm(test_loader, desc='[test]', leave=False) for image_id, image in pbar: image_id, image = image_id, image.to(device) sum_ai_probs = None sum_style_probs = None for m in models: logit_ai, logit_cls = m(image) probs_ai = F.softmax(logit_ai, dim=1) probs_cls = F.softmax(logit_cls, dim=1) if sum_ai_probs is None: sum_ai_probs = probs_ai.clone() sum_cls_probs = probs_cls.clone() else: sum_ai_probs += probs_ai sum_cls_probs += probs_cls mean_ai_probs = sum_ai_probs / len(models) mean_cls_probs = sum_cls_probs / len(models) ai_pred = mean_ai_probs.argmax(dim=1).cpu().tolist() cls_pred = mean_cls_probs.argmax(dim=1).cpu().tolist() pred_ai.extend(ai_pred) pred_style.extend(cls_pred) ids.extend([int(x) for x in image_id]) </pre>

KFold Validation 하이퍼파라미터 탐색을 통해 최적 하이퍼파라미터를 선정하였습니다. 이번 모델에서는 가장 낮은 validation loss를 보이는 파라미터와 가장 높은 style accuracy를 보이는 파라미터를 바탕으로 성능 극대화를 위해 기존 학습과 검증에 사용했던 데이터 전체를 다시 합쳐 **full_dataset**으로 구성한 뒤 모델을 재학습 하였습니다. 사용한 배치 크기 및 옵티마이저 등은 KFold에서 사용했던것과 동일합니다. 학습은 113 epoch 수행하였으며 단일 Train/Validation 학습 및 KFold을 통해 얻어낸 가장 안정적으로 수렴한다 판단되는 epoch 구간입니다. 학습 완료 후 최종 파라미터는 별도 파일로 저장하였습니다.

테스트 데이터에 대한 예측 성능을 향상시키기 위해 양상불 기법을 사용하였습니다. 가장 최적의 하이퍼파라미터라고 판단된 두 가지 설정을 사용하여 두 모델을 학습시켰고, 두 모델을 묶어 모델들의

출력 확률을 평균을 내어 클래스를 분류하도록 하였습니다. 결과물을 출력하는 함수는 `baseline.ipynb`에서 기본적으로 제공된 함수에서 앙상블 모델에 맞도록 일부 변경하였습니다.

결과적으로 두 설정을 사용한 두 가지 모델만을 사용하여 제출하였고, 다음과 같은 결과를 얻을 수 있었습니다.

20215254_최기현		0.67142	14	11h
--------------	---	---------	----	-----

Step 8: Training Techniques

성능 개선을 위해서 사용한 기법 중에서 특별히 효과적이었던 부분이나 강조하고자 하는 내용을 작성해주세요.

여러 학습 기법중 이미지 변환같은 데이터 증강과 Batch Normalization도 효과적이었지만, CosineAnnealingLR은 특히나 성능이 크게 체감되었을 정도로 효과적이었습니다. lr이 점진적으로 감소되어 epoch이 늘어날수록 스케줄러를 사용하지 않았을때와는 비교도 되지 않을 정도로 안정적으로 수렴되는것을 볼 수 있었습니다.

하지만 결과적으로 가장 크게 기여한 기법은 KFold Validation과 앙상블 기법이었습니다. KFold Validation은 코드를 구현하는것이 매우 복잡했고 학습의 시간도 매우 오래 걸렸지만 최적 하이퍼파라미터를 찾아내고 안정적인 일반화 성능을 만들어 내기에 가장 좋은 방법이었습니다. 앙상블 기법은 복수 모델을 사용함으로써 편향을 줄이고 안정적인 예측을 얻을 수 있도록 하여 최종 점수 반영에 큰 역할을 해내었습니다.

이번 과제는 상당히 다양한 학습 기법을 사용해 볼 수 있었던 기회였습니다. 거의 대부분이 기법이 validation loss를 가시적으로 줄어들게 해주었습니다. 더 다양한 하이퍼파라미터와 다른 Neural Network를 시도해보지 못함에 조금은 아쉬움이 들기도 합니다.
